

Decomposed Clearing via Fisher Market Budget Allocation

Prediction Market Clearing through Eisenberg-Gale Decomposition

Draft — February 2026

Summary. Prediction market batch auctions with bundle orders face an exponential state space: N groups of mutually exclusive outcomes produce $\prod K_j$ joint states. We observe that the Fisher market (Eisenberg-Gale) formulation of clearing provides a natural decomposition mechanism that the standard linear welfare formulation lacks. The key insight: with log utility, splitting a market maker’s budget across independently-solved components is a *smooth concave* optimization problem, whereas with linear welfare the same budget split is *combinatorial*. We prove that the optimal split equalizes utility across components, and that standard mirror descent converges to it at rate $O(1/t)$ with each iteration requiring only independent per-component solves.

1. Introduction

The companion paper (*Prediction Markets Are Fisher Markets*) establishes that batch auction clearing with budget-constrained market makers is an Eisenberg-Gale convex program. This paper addresses the computational consequence: the joint state space is exponentially large, and we need to solve the clearing problem anyway.

The problem. Consider N groups of mutually exclusive outcomes, group j having K_j outcomes. The joint state space is $\mathcal{S} = \prod_j \{1, \dots, K_j\}$, with $|\mathcal{S}| = \prod K_j$. A *bundle order* is an order whose payoff depends on the joint state: “buy YES on both A and B ” pays \$1 only in the joint state where both resolve YES. Such orders create non-separable demand across groups.

When no orders span multiple groups, the clearing problem decomposes into independent per-group programs — the product-LMSR factorization (Chen and Pennock 2007). Cross-group orders break this, coupling groups *transitively*: if orders link A – B and B – C , clearing requires the full $K_A \times K_B \times K_C$ state space. In practice, even a handful of bundle orders can connect all groups into one giant component.

The core insight. The Eisenberg-Gale structure provides a decomposition mechanism that the linear (risk-neutral) formulation lacks. With log utility, an MM with budget B_k and orders across multiple components can have its budget *smoothly allocated* across components, with each component solving independently. The optimal allocation satisfies a clean condition — equal utility across components — and standard mirror descent finds it. With linear welfare, the same budget allocation is a combinatorial problem: a hard constraint $\text{cap}_k \leq B_k$ cannot be smoothly split.

What this paper proves.

1. The *budget decomposition theorem*: when no orders span multiple components, the joint EG program decomposes into independent per-component programs coordinated by budget allocation. The optimal allocation equalizes utility across components (§2).
2. *Mirror descent convergence*: the iterative budget reallocation algorithm converges at rate $O(1/t)$, with each iteration requiring only independent per-component solves (§3).

What this paper does not prove. Tight welfare bounds for the case when bundle orders *do* span components. We discuss the approximation quality of dropping or leg-decomposing cross-component orders (§4), but the bounds are loose. We also sketch directions for exploiting coupling graph structure and external solvers (§5), without proofs.

2. Budget Decomposition

2.1. Setup

We use the notation and results of the companion paper. The risk-averse batch auction clearing program over a joint state space $\mathcal{S}$ is the quasi-linear Fisher market:

$$P^{\text{RA}} : \max_{\mathbf{q} \in \mathcal{C}, \mathbf{s} \geq 0} \sum_k [B_k \ln(U_k(\mathbf{q}) + s_k) - s_k] + \sum_{j \notin \text{MM}} w_j q_j - C_b(\mathbf{D}(\mathbf{q}))$$

where $\mathbf{D}(\mathbf{q}) \in \mathbb{R}^{|\mathcal{S}|}$ is the net demand vector over joint states, $C_b(\mathbf{D}) = b \ln \sum_{s \in \mathcal{S}} \exp(D_s/b)$ is the smoothed minting cost, $U_k(\mathbf{q}) = \sum_{i \in \text{MM}_k} L_i q_i$ is MM k 's weighted fill, and $s_k \geq 0$ is retained cash.

We focus on the *capital-constrained* regime where $s_k = 0$ for all MMs (i.e., $\mu_k = B_k/U_k > 1$). When budgets don't bind, the LP solution is optimal regardless of budget allocation, so decomposition is trivially exact.

Suppose the groups partition into *components* $\mathcal{M} = \{C_1, \dots, C_M\}$. Each component C_m has its own state space $\mathcal{S}_m = \prod_{j \in C_m} \{1, \dots, K_j\}$, orders $\mathcal{O}_m$, and minting cost C_b^m .

2.2. The Monolithic vs. Decomposed Program

If an MM k has orders in multiple components, the monolithic program optimizes over all of them jointly. The decomposed approach allocates a *budget share* $B_k^m \geq 0$ to each component, with $\sum_m B_k^m = B_k$, and solves each component independently:

$$P_m(\mathbf{B}^m) : \max_{\mathbf{q}_m \in \mathcal{C}_m} \sum_k B_k^m \ln U_k^m(\mathbf{q}_m) + \sum_{j \in \mathcal{O}_m \setminus \text{MM}} w_j q_j - C_b^m(\mathbf{D}^m(\mathbf{q}_m))$$

The *coordination problem* is to find the budget allocation $\mathbf{B} = \{B_k^m\}$ that maximizes total welfare:

$$\max_{\mathbf{B} \geq 0} \sum_m W_m^*(\mathbf{B}^m) \quad \text{s.t.} \quad \sum_m B_k^m = B_k \quad \forall k$$

where $W_m^*(\mathbf{B}^m)$ is the optimal welfare of component m given budgets $\mathbf{B}^m$.

Theorem 1 (Budget Decomposition). *When no orders span multiple components, the decomposed program with optimal budget allocation achieves the same welfare as the monolithic program. The optimal allocation satisfies, for each MM k active in components m and m' :*

$$\ln U_k^{m*} = \ln U_k^{m'*}$$

Equal utility across components: MM k achieves the same weighted fill in every component where it is active.

Proof. When no orders span components, the minting cost separates: $C_b = \sum_m C_b^m$ (product-LMSR). We prove the claim in two steps: (i) the optimal budget allocation equalizes utility, and (ii) the decomposed fills match the monolithic optimum.

(i) Equal utility. Each per-component program $P_m(\mathbf{B}^m)$ is a strictly concave maximization with a unique optimum. The objective $B_k^m \ln U_k^m(\mathbf{q}_m)$ is jointly concave in

$(B_k^m, \mathbf{q}_m)$ (it is the perspective of $\ln U_k^m$, and the perspective of a concave function is concave). By the envelope theorem, the marginal value of budget in component m is:

$$\frac{dW_m^*}{dB_k^m} = \ln U_k^{m*}$$

(The indirect effect through optimal fills vanishes by the optimality of $\mathbf{q}_m^*$.) The coordination problem $\max_{\mathbf{B}} \sum_m W_m^*(\mathbf{B}^m)$ subject to $\sum_m B_k^m = B_k$ has the first-order condition $\ln U_k^{m*} = \lambda_k$ for all active components: MM k achieves the same utility in every component where it holds budget.

(ii) **Fills match the monolithic.** The monolithic program has shadow price $\mu_k = B_k / \sum_m U_k^{m*}$ — the same for all components. At the decomposed optimum, $U_k^{m*} = u_k$ for all active m , so $\sum_m U_k^{m*} = M_k u_k$ (where M_k is the number of active components). The decomposed shadow price $\mu_k^m = B_k^m / u_k = (B_k / M_k) / u_k = B_k / (M_k u_k) = \mu_k$.

Since shadow prices match and minting costs are independent, the KKT conditions for each order i are identical in both programs. By uniqueness of the per-component optimum, the fills agree. $\square$

Remark. The decomposed objective $\sum_m B_k^m \ln U_k^m$ is *less* than the monolithic objective $B_k \ln \sum_m U_k^m$ (by Jensen’s inequality). This reflects the pooling benefit of joint Kelly optimization. The theorem says the *fills* agree, not the objective values.

Remark. This theorem is most interesting when MMs have orders across multiple independently-solvable components. Without cross-component MMs, the components are fully independent and no coordination is needed at all.

2.3. Why Linear Welfare Cannot Decompose

In the risk-neutral (linear welfare) model, MM k ’s contribution is $\sum_{i \in \text{MM}_k} w_i q_i$ with a hard budget constraint $\text{cap}_k \leq B_k$. There is no natural way to split a hard budget across components: the constraint $\text{cap}_k^m \leq B_k^m$ with $\sum_m B_k^m = B_k$ introduces a combinatorial allocation problem (which component gets how much budget?) on top of the already-intractable clearing problem. The $\ln$ utility replaces this discrete allocation with a smooth, differentiable one. This is the computational payoff of the Fisher market structure.

3. Budget Equalization via Mirror Descent

The equal-utility condition (Theorem 1) suggests a natural iterative algorithm. The coordination problem $\max \sum_m W_m^*(\mathbf{B}^m)$ subject to $\sum_m B_k^m = B_k$ is concave, and its gradient is $\partial W_m^* / \partial B_k^m = \ln U_k^{m*}$ (envelope theorem). Mirror descent with KL divergence gives:

1. **Initialize.** For each MM k , set $B_k^m = B_k / |\{m : \text{MM}_k \cap \mathcal{O}_m \neq \emptyset\}|$ (equal split across active components).
2. **Solve.** For each component m in parallel, solve $P_m(\mathbf{B}^m)$ to get optimal fills $\mathbf{q}_m^*$ and utilities U_k^{m*} .
3. **Update.** For each MM k , reallocate:

$$B_k^m \leftarrow B_k \cdot \frac{B_k^m \cdot U_k^{m*}}{\sum_{m'} B_k^{m'} \cdot U_k^{m'*}}$$

Multiply each component’s allocation by its utility and renormalize.

4. **Repeat** steps 2–3 until convergence.

The update is *multiplicative weights*: each component’s budget share grows proportionally to the utility it delivers. At the fixed point, U_k^{m*} must be constant across m , matching Theorem 1.

Theorem 2 (Convergence). *The mirror descent algorithm converges to the optimal budget allocation. For smooth W_m^* , the welfare gap satisfies $W^* - W(\mathbf{B}^t) = O(1/t)$.*

Proof sketch. The coordination problem is concave (each W_m^* is a concave value function). The update exponentiates the gradient and renormalizes — mirror descent with KL divergence. The $O(1/t)$ rate for concave maximization over the simplex is standard (Beck and Teboulle 2003). Smoothness follows from continuous dependence of equilibrium utilities on budgets. $\square$

Remark. Each iteration requires solving M independent convex programs — fully parallelizable. In practice, 3–5 iterations suffice for budget stabilization. Interior-point solvers can warm-start from the previous iteration’s solution.

Practical speed. Without bundle orders, the decomposition gives a significant speed advantage. If N groups each have K outcomes, the monolithic solve has state space K^N while the decomposed solve runs N independent K -state programs in parallel. Even for moderate N and K , this is the difference between intractable and instant. With MMs spanning components, add 3–5 iterations of budget equalization — still far cheaper than monolithic.

4. Welfare Bounds for Approximate Decomposition

When bundle orders span multiple components, the decomposition is not exact and cross-component orders must be handled approximately. We state two simple bounds; both are loose but provide the right qualitative picture.

Dropping cross-component orders. Excluding all cross-component orders $\mathcal{O}_\times$ loses at most $\sum_{i \in \mathcal{O}_\times} w_i \bar{q}_i$ welfare (sum of each dropped order’s limit price times max fill). This is an immediate consequence of feasibility containment.

Leg decomposition. Decomposing each bundle into per-component marginal legs (averaging over the other components’ states at reference prices) introduces error bounded by the *interaction magnitude* $\Delta_i = \|\varphi_i - \hat{\varphi}_i\|_\infty$: the welfare gap is at most $\sum_{i \in \mathcal{O}_\times} \bar{q}_i \Delta_i$. This follows from the Lipschitz property of the smoothed minting cost.

Limitations. Both bounds are loose in practice. The drop bound ignores that cross orders compete for minting capacity with within-component orders. The leg decomposition bound is “tight when $\Delta_i \approx 0$ ” — but this is vacuous, since the interesting cases have $\Delta_i = 1/4$ (buy both YES) or $\Delta_i = 1/2$ (conditional orders). Tighter instance-dependent bounds remain open.

5. Directions

This section sketches computational and architectural ideas that we have not proved but believe are promising.

5.1. Coupling Graph and Structured Computation

The *coupling graph* has market groups as vertices and edges wherever a bundle order creates non-separable dependence. Connected components identify the minimal sets of groups that must be solved jointly. In principle, if the coupling graph has low treewidth τ , minting cost gradients can be computed in $O(K^{\tau+1})$ time via junction tree algorithms rather than $O(K^N)$ full enumeration.

In practice, coupling graphs may not have low treewidth — a few popular bundles can connect many groups transitively. The question is whether *demand sparsity* helps: most joint states have negligible demand, so column generation (iteratively discovering active states) may avoid full enumeration even when the coupling graph is dense.

5.2. A Searcher Model

The decomposition framework suggests a *market for clearing services* analogous to Proposer-Builder Separation in Ethereum. The exchange solves per-group clearing (the easy separable part) and coordinates budget allocation. External *searchers* compete to solve coupled components: a searcher takes a set of coupled groups, solves the joint clearing problem, and submits fills and prices. The exchange selects and combines proposals via budget equalization.

The log-utility structure makes this naturally proportional rather than winner-take-all: multiple searchers’ solutions can coexist, weighted by the welfare they deliver. Whether this is incentive-compatible, collusion-resistant, or practically useful remains to be established.

6. Discussion

6.1. What We Proved

Setting	Linear welfare	Log welfare (EG)
Single group	LP (easy)	Convex program (easy)
Multi-group, no bundles	Independent LPs	Independent EGs + budget split
Multi-group + cross-component MMs	Combinatorial budget split	Smooth budget split (mirror descent)

The key result is the contrast in the last row: with linear welfare, splitting an MM’s hard budget constraint across components is a combinatorial optimization. With log utility, the same problem is smooth and concave — the budget flows to where it generates the most utility, equalized at optimum by a simple multiplicative-weights iteration.

6.2. What Remains Open

1. **Tight welfare bounds for bundles.** Our bounds for dropping or leg-decomposing cross-component orders are worst-case. Instance-dependent bounds would be much more useful for deciding when approximate decomposition is acceptable.
2. **Bundle handling.** The per-market leg decomposition used by our LP and EG solvers misprices non-separable bundles. The fill price is linear in per-market prices ($0.5p_A + 0.5p_B$ for “buy both YES”) while the true value is multiplicative ($p_A \cdot p_B$). At $p_A = 0.6, p_B = 0.4$: leg price = 0.50, true price = 0.24. This overpricing can reject profitable bundle orders. Correctly pricing bundles requires solving the joint-state formulation, making the decomposition approach essential.
3. **Practical convergence.** We proved $O(1/t)$ worst-case. Empirical convergence may be much faster. Characterizing when this happens would guide the choice between decomposition and monolithic solving.

This paper is a companion to Prediction Markets Are Fisher Markets (2026), which establishes the Eisenberg-Gale structure that this paper exploits computationally.